M0 / R1 — parity harness: C3's transformer in PyTorch vs Flax NNX

jax-experiments writeup · M0

2026-10-15

Contents

Question **2**

The idea **2**

The math **2**

What we measure and why **2**

What I built **3**

Results **3**

What surprised me **5**

Open threads **5**

Status: done, on CPU · **Data:** `data/r1-parity.json` ·
Reproduce: `uv sync --extra torch && uv run python experiments/r1_parity/run.py` (about 72 s on 4 CPU cores; needs an ml-ladder checkout at `../ml-ladder` or `--ml-ladder`) ·
Regression test: `uv run pytest tests/test_r1_parity.py`

Question

Does the Flax NNX port compute the same function as C3's PyTorch transformer, which every Track R number comes from? And which silent mismatches would the comparison catch?

The idea

This is ml-ladder G3's method, pointed at JAX instead of Candle. Export the PyTorch weights, load them into the port, feed both the same input, and diff the outputs. The reference is **ml-ladder's own code, imported from the checkout** (commit `42489de`), not a copy. That means C3's `make_transformer` for the block, and `tools/gpu/c_context_scaling.transformer_lm` for the SDPA and RoPE models R2 will train.

The math

The verdict is the relative error and the largest absolute difference:

$$\varepsilon_{\text{rel}} = \frac{\|F_{\text{jax}}(x) - F_{\text{torch}}(x)\|}{\|F_{\text{torch}}(x)\|}, \qquad \max_i |F_{\text{jax}}(x)_i - F_{\text{torch}}(x)_i|.$$

As in G3, the size of the gap diagnoses its cause:

- **About** 10^{-7} **in fp32** means the same arithmetic done in a different order.
- 10^{-6} **to** 10^{-3} means a different function: a swapped approximation or a different epsilon.
- **Order** 10^{-1} means a layout error.

What we measure and why

Block. G3's exact settings: width 64, 4 heads, length 16, seeds 0–2, and inputs drawn with `torch.Generator().manual_seed(seed + batch)`. That makes the Candle and JAX numbers directly comparable. Batch 1 *and* batch 4, because layout bugs can hide at batch 1. The arms:

- **Matched**: PyTorch numerics, explicit softmax attention.
- **Fused attention**: `jax.nn.dot_product_attention` in place of the explicit form.
- **Silent defaults, one at a time.** Three places where Flax or JAX *default* to something different from PyTorch, none of which raises an error:
  - `jax.nn.gelu` is the **tanh approximation** (`approximate=True`), while `torch.nn.GELU()` is exact;
  - `nnx.LayerNorm` uses $\varepsilon = 10^{-6}$, while PyTorch uses 10^{-5};
  - `nnx.LayerNorm` computes **fast variance** $\mathrm{E}[x^2] - \mathrm{E}[x]^2$, while PyTorch subtracts the mean first.

  Then all three together, which is what a naive port computes.
- **Fast variance with the input shifted by +100.** $\mathrm{E}[x^2] - \mathrm{E}[x]^2$ cancels catastrophically when the mean is large relative to the spread.

- **The square output projection loaded untransposed.** `nn.Linear` stores [out, in] and NNX stores [in, out]. A non-square layer fails with a shape error; a square one silently computes something else.
- **`jax.default_matmul_precision("highest")`** against the default. PLAN.md flagged TF32. TF32 cannot occur on a CPU, nor on a T4 (sm_75 has no TF32), so this is a control.

Whole models: R2's configurations, against the SDPA forward R2 actually used:

- the 4M transformer with learned positions;
- the 4M transformer with RoPE;
- the 25M transformer with RoPE at T = 1024.

Gradients: next-byte cross-entropy on the 4M model at T = 256, batch 4. Compared are the loss and every parameter's gradient (both the global relative norm and the worst single tensor). A training run can only reproduce if the backward pass agrees too.

What I built

- **`src/jaxexp/transformer.py`**: C3's `CharTransformer` and its pre-LN block in Flax NNX.
  - Positions can be learned, sinusoidal, none, or RoPE (the half-split form from `c_context_scaling.rotary`, rotated in fp32).
  - Attention can be explicit or fused.
  - A `Numerics` dataclass holds the three disputed defaults. **Its default is PyTorch's**, so a model built without options computes the reference function. `FLAX_DEFAULTS` gives the naive port.
- **`src/jaxexp/torch_port.py`**: loads a `state_dict` (as NumPy, so torch is never imported) into the NNX model. It transposes `Linear` weights and maps the names (`weight` → `scale/embedding`, `mlp.0/mlp.2` → `up/down`). It is strict both ways: every torch tensor must land somewhere, and every NNX parameter must be assigned.
- **`experiments/r1_parity/run.py`**: the harness. It writes the JSON with versions and the ml-ladder commit.
- **`tests/test_r1_parity.py`**: 17 tests.
  - **Always run (JAX only)**: the causal mask does not leak, for every position type × attention path; the RoPE score depends only on offset; the loader is strict.
  - **Need torch and ml-ladder**: block parity $< 10^{-6}$ at batch 1 and 4, with both attention paths; 4M model parity $< 10^{-6}$ with learned and RoPE positions; and a check that the harness *does* flag tanh GELU, $\varepsilon = 10^{-6}$, and the untransposed projection. Without that last check, a passing parity test would prove nothing.

Results

Everything below was run on CPU (4 cores, x86_64) with Python 3.12.3, jax 0.11.2, flax 0.12.10 and torch 2.14.1+cpu. Values are medians over seeds 0–2, with min–max in parentheses.

The block, at G3's settings:

arm	relative, batch 1	relative, batch 4	max |diff|, batch 4	vs matched
matched	**7.86e-08** (7.74e-08–7.99e-08)	**7.69e-08** (7.56e-08–7.86e-08)	4.77e-07	1×

arm	relative, batch 1	relative, batch 4	max |diff|, batch 4	vs matched
fused attention	7.86e-08 (identical)	7.69e-08 (identical)	4.77e-07	1×
tanh GELU (`jax.nn.gelu` default)	3.60e-05 (3.58e-05–3.72e-05)	3.93e-05 (3.86e-05–3.97e-05)	1.43e-04	510×
LayerNorm ε 1e-6 (Flax default)	1.43e-06 (1.27e-06–1.55e-06)	1.44e-06 (1.42e-06–1.53e-06)	6.79e-06	18.7×
fast variance (Flax default)	7.88e-08 (7.82e-08–8.14e-08)	7.74e-08 (7.51e-08–8.05e-08)	4.77e-07	1.01×
all three Flax/JAX defaults	3.52e-05 (3.52e-05–3.66e-05)	3.86e-05 (3.79e-05–3.89e-05)	1.40e-04	501×
matched, input +100	3.77e-08 (3.70e-08–4.06e-08)	3.27e-08 (3.20e-08–3.58e-08)	1.53e-05	0.43×
fast variance, input +100	2.79e-06 (2.57e-06–3.27e-06)	1.37e-06 (1.25e-06–1.38e-06)	8.16e-04	17.8×
projection untransposed	2.34e-01 (1.99e-01–2.54e-01)	2.40e-01 (2.01e-01–2.41e-01)	1.60e+00	3.1×10^6
matmul precision "highest"	7.86e-08 (identical)	7.69e-08 (identical)	4.77e-07	1×

For scale, PyTorch disagrees with *itself* by more than that. Its own explicit attention and its SDPA kernel differ by **1.27e-07** (1.15e-07 to 1.37e-07) on the same blocks and inputs.

Against ml-ladder G3 (its JSON, `g-metrics.json`, from the Candle port):

	G3: Candle	R1: JAX
matched, batch 1	8.36e-08 (8.06e-08–8.91e-08)	7.86e-08 (7.74e-08–7.99e-08)
tanh GELU, batch 1	3.60e-05 (3.58e-05–3.72e-05)	3.60e-05 (3.58e-05–3.72e-05)
max |diff|, matched	4.77e-07	4.77e-07

Whole models (logits), and the gradient:

model	parameters	explicit, batch 1	explicit, batch 4	fused, batch 4
4M, learned positions, T = 256	4,145,920	3.37e-07 (3.37e-07–3.41e-07)	3.38e-07 (3.38e-07–3.41e-07)	3.38e-07 (identical)
4M, RoPE, T = 256	4,080,384	3.93e-07 (3.93e-07–3.98e-07)	3.97e-07 (3.96e-07–3.98e-07)	3.97e-07 (identical)
25M, RoPE, T = 1024	25,482,240	6.14e-07 (6.12e-07–6.19e-07)	6.13e-07 (6.12e-07–6.15e-07)	6.13e-07 (identical)

Gradient check (4M model, learned positions, T = 256, batch 4):

- **Loss:** relative error **8.31e-08** (8.31e-08–8.35e-08).
- **All gradients together:** relative error **4.87e-07** (4.86e-07–4.92e-07).
- **Worst single tensor:** a first-LayerNorm scale every seed (blocks 0, 3 and 0), at 7.8e-07 to 8.4e-07.

Verdict:

- With matched settings, the port agrees with C3 to fp32 round-off. That holds for one block, for whole models up to 25M parameters at T = 1024, and for the backward pass.
- The error grows with depth: 7.7e-08 for one block, 3.4e-07 for 5 layers, 6.1e-07 for 8 layers at 4× the context. It stays under the 10^{-6} the tests assert.

What surprised me

The tanh-GELU error is a property of the function, not of the port.

- JAX's number matches Candle's from G3 almost exactly: the same median, min and max to three significant figures, from two unrelated implementations.
- At 3.6×10^{-5} the approximation error is about 500× round-off, so it dominates everything else.
- So G3's "470× worse" and this "510× worse" are not two measurements of different bugs. They are one fact about tanh against Φ at these activations.

JAX's own default is the trap. In G3 the trap was Candle's `gelu()`. Here it is `jax.nn.gelu`, whose default is `approximate=True`. A port that writes the obvious line computes a different function from `torch.nn.GELU()` and raises nothing. Flax's two LayerNorm defaults both differ from PyTorch too.

Fast variance costs nothing until the mean is large.

- On zero-mean inputs it is indistinguishable from the two-pass form (1.01×).
- Shift the input by +100 and it is 18× worse: 1.4×10^{-6} relative, 8×10^{-4} in absolute terms.
- Trained residual streams do develop large-magnitude channels. So matching PyTorch's two-pass form is the safe default for R2, even though random-init parity can't tell them apart.

PyTorch's SDPA is further from PyTorch's explicit attention than JAX is. Different kernels in one framework disagree by 1.3×10^{-7}. The cross-framework port disagrees by 7.7×10^{-8}. Round-off has no allegiance to a framework.

Two checks were void on this hardware, which is a result in itself.

- On CPU, `jax.nn.dot_product_attention` lowers to the same XLA computation as the explicit form. The outputs are *bitwise* identical, even for a 512-wide RoPE block at T = 1024 and batch 4. (That last check was run ad hoc and is not in the JSON.) So "fused vs explicit" says nothing here.
- `matmul_precision="highest"` changes nothing either: there is no TF32 path on a CPU.

Both questions move to the GPU.

Open threads

- **The fused path on the T4.** Rerun the fused/explicit arms in a Kaggle session to find which kernel `implementation=None` picks on sm_75, and how far it lands from the explicit form.

cuDNN's flash attention needs sm_80+, so it is probably XLA's own lowering.

- **fp16.** All of the above is fp32. R2 trains under PyTorch's fp16 autocast, and Flax has no autocast, so R2 needs an explicit choice: which ops run in fp16, with the loss and softmax kept in fp32. The next parity check is the port in fp16 against PyTorch's autocast forward on the T4.
- **Trained weights.** These are random-init weights, as in G3. Trained weights have larger dynamic range and the large-mean channels that expose fast variance. Re-run the harness on R2's first trained checkpoint.
- **Training-step parity.** Logits and gradients agree. Optax's `adamw` + `clip_by_global_norm` + the warmup-cosine schedule against `torch.optim.AdamW` over a few steps is the last piece before R2's numbers can be compared step for step.